# SYSTEM ANALYSIS DOCUMENTATION (SAD)

**UMHackathon 2026**

**Four Fingers**

---

**Table of Contents**

1. Introduction 3

2. System Architecture & Design 7

3. Functional Requirements & Scope 13

4. Monitor, Evaluation, Assumptions & Dependencies 15

5.Project Management & Team Contributions 17

# 1. Introduction

TalentFlow is a centralized active application, recruitment and onboarding platform designed to address the inefficiencies found in traditional hiring workflows. Current recruitment processes often are fragmented across spreadsheets, emails and multiple job platforms with little to no transparency making it difficult for Human Resource (HR) teams to manage candidate data effectively, for applicants to track their applications and for organisations to be held accountable over transparent hiring processes.

This system introduces an integrated digital solution that consolidates candidate intake, application tracking and onboarding all compiled to a single platform easily integradable to any organisation. By incorporating AI-powered analysis through Z.Ai's GLM. Talentflow enhances decision making and credibility by evaluating prospective candidate suitability, validating submissions and generating actionable insights for recruiters.

## *1.1. Purpose:*

This system analysis document outlines the technical design, architecture, and operational workflow of Talentflow. It provides a clear understanding of how the system is structured, how data flows across components, and how core functionalities are implemented. This document serves as a reference for developers, testers, and stakeholders to ensure alignment in system design, implementation and future scalability.

So, the key element of system that has been covered are as followed:

1. **Architecture**:
    a. **Frontend Layer**
        - Uses CSS, HTML, and JavaScript
        - Interactive dashboards powered by Chart.js
        - Provides HR Dashboards, candidate views, monitoring interfaces.
    b. **Backend Layer**
        - Uses FastAPI
        - Handles API request, business logic, authentication
        - Bridge between frontend - database
    c. **Database**
        - Uses Supabase
        - Stores all records involving candidate data, application, analytics and employee data.
    d. **Ai Processing**
        - Z.Ai GLM integration
        - Performs resume parsing, validation, and candidate suitability scoring
        - Generate insights and summaries
    e. **Deployment Model**
        - Can be deployed as a cloud based system, supports scalability.
2. **Data Flows:**

The data flow in TalentFlow ensures seamless interaction between users and system components:

a. **Candidate Submission**
   - Candidate Submits application via form and CV
   - Data is sent to backend API
b. **Data Processing**
   - Backend validates input and stores in Supabase
   - Ai model analyzes resume and generates
     i. suitability score
     ii. extracted information
     iii. insights
c. **Storage**
   - All processed data is stored in the Supabase database
d. **Hr Dashboard Display**
   - Frontend fetches processed data via Supabase callbacks and API
     Displays:
     - Application statistics
     - Pipeline stages
     - Department breakdown
     - Ai insights
e. **Real Time Updates**
   - Any and all updates are reflected dynamically.

3. **Model Process:**
   The system workflow represents the end-to-end recruitment lifecycle
   a. **Application intake**
      - Candidates submit applications with required documents
   b. **Ai processing**
      - GLM extracts key data (skills, experience)
      - Calculates recommendation/suitability score
   c. **Pipeline management**
      - Candidates moves through stages:
      - New - reviewing - interview - offer - onboarding
   d. **Monitoring**
      - HR monitors progress via dashboard:
      - Statistics
      - Charts
      - Candidate tables
      - AI insights
   e. **Onboarding tracking**
      - Accepted candidates are tracked with start dates
      - Calendar and onboarding status updated

4. Role of Reference:
   This document serves as a **technical and functional reference** for:
   - Developers → understanding system structure and API flow
   - QA Team → validating expected behavior and edge cases

- Stakeholders → understanding system capabilities and impact

It ensures consistency in development and provides a foundation for future enhancements.

1.2. **Background**:

Organizations require a unified system that automates and organizes recruitment workflows while improving decision-making efficiency. Traditional recruitment systems rely heavily on manual processes such as spreadsheets, emails, and disconnected platforms. This leads to:

- Inefficient tracking of candidate applications
- Increased risk of human error
- Delayed hiring processes
- Poor candidate experience

1. Previous Version:

   Previously, recruitment processes were:
   - Manual and decentralized : "Data stored across multiple tools (emails, spreadsheets)"
   - Non-standardized : "Non-standardized"
   - Time-consuming : "Manual verification and tracking"

2. Changes in Major Architectural Components

   TalentFlow introduces several key improvements:
   - **Centralized Platform**
     - All recruitment activities in one system
   - **AI Integration**
     - Automated resume analysis and scoring
   - **Real-Time Dashboard**
     - Live monitoring of applications and onboarding
   - **Structured Data Handling**
     - Standardized forms and database storage
   - **Interactive Visualizations**
     - Charts, pipelines, and insights for better decision-making

3. New capabilities that are only introduced in this
   - AI-powered candidate suitability scoring
   - Automated data extraction from resumes
   - Real-time recruitment pipeline visualization
   - Centralized onboarding tracking system
   - Calendar-based onboarding scheduling
   - Insight generation for HR decision support

### 1.3. Target Stakeholder

| Stakeholders | Roles | Expectations |
|---|---|---|
| HR / Recruitment Team | Manage applications, track candidates, oversee hiring pipeline | Efficient dashboard, accurate insights, simplified workflow |
| Managers | Monitor hiring progress and make decisions | Clear analytics, quick access to candidate data |
| Candidates | Submit applications and track progress | Simple application process, transparency |
| Development Team | Build and maintain the platform | Modular codebase, clear API structure |
| QA Team | Test system functionality and performance | Defined workflow, edge case coverage |

# 2. System Architecture & Design

## 2.1. High Level Architecture

### 2.1.1. Overview:

| Type | Details |
|---|---|
| System | Web based Application (Browser-based HR & Job Application Platform) |
| Architecture | Cloud-based Client–Server Architecture with AI Service Layer |

The system is structured as a **client-server-based web application** consisting of two main user roles: **Candidates (Employees)** and **Employers (HR/Admins)**.

The **frontend (browser-based interface)** allows candidates to upload CVs, submit job applications, and track their application status, while employers can view applications, manage candidate pipelines, and access analytics dashboards.

All client interactions are handled through a **centralized backend API built with FastAPI**, which acts as the core processing layer of the system.

The backend is responsible for:

- Handling authentication via Supabase Auth
- Processing CV uploads and extracting text
- Communicating with the AI service (GLM) for CV parsing and candidate evaluation
- Managing application data and business logic
- Generating analytics for the HR dashboard

The system integrates an **AI Service Layer (GLM via OpenRouter API)**, which is treated as an external microservice responsible for:

- Converting unstructured CV data into structured JSON
- Scoring and analyzing candidates
- Generating intelligent log insights

All persistent data is stored in a **PostgreSQL database (hosted on Supabase)**, which maintains structured records for users, applications, application statuses, and system logs.

The architecture follows a **modular and service-oriented approach**, where key functionalities such as CV parsing, application processing, dashboard analytics, and logging are logically separated within the backend, enabling scalability and maintainability.

Overall, the system demonstrates a **distributed architecture** where:

- The **frontend** handles user interaction
- The **backend (FastAPI)** manages logic and API communication
- The **AI service (GLM)** provides intelligent processing
- The **database (Supabase/PostgreSQL)** ensures persistent storage

### 2.1.2 LLM as Service Layer

TalentFlow integrates the Z.AI GLM model as a dedicated AI service layer within the backend rather than treating it as a standalone module. The backend (FastAPI) acts as the orchestrator, controlling how data is prepared, sent to the model, and how responses are validated and stored.

**How Prompts are Constructed**
Prompts are dynamically generated in the backend varying between tasks.

- **CV Parsing**
  - A strict system instruction is used to enforce structured JSON output
  - The extracted PDF text is truncated (raw_text[:4000]) to fit token limits
  - Categories (department, employment type, etc.) are constrained to predefined values
- **Candidate evaluation**
  - A simplified prompt is constructed using role title and skills
  - The model returns structured fields such as: “suitability score” and “justification text”
- **System Monitoring**
  - Prompts summarize system events
  - Output is constrained to short JSON logs (category + insight)

**Context Window Handling**
The system controls what is sent to the model to avoid excessive token usage:
- Only relevant CV text is included (first ~4000 characters)
- Prompts are task-specific (parsing vs evaluation vs logging)
- Structured instructions reduce unnecessary output verbosity

This ensures:
- Faster response time
- Lower API cost
- More deterministic outputs

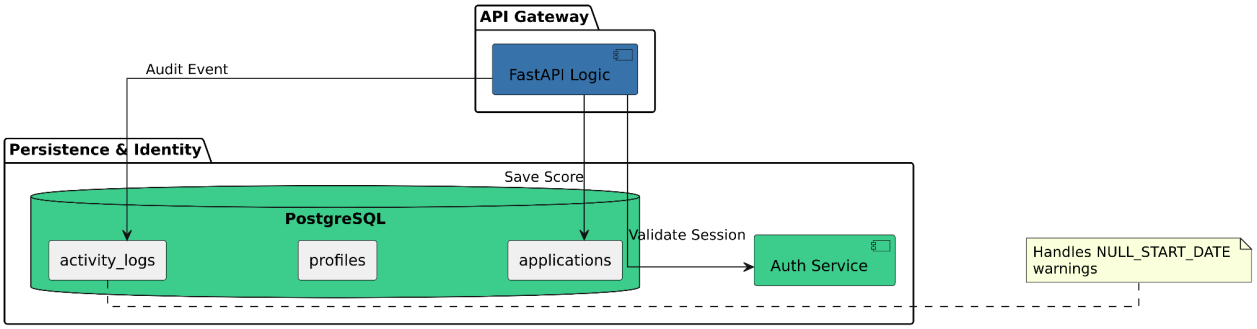
**Response Handling & Parsing**
After receiving responses from GLM
- Raw output is extracted from the API response
- Markdown/code block formatting is removed if present
- JSON is parsed using json.loads()
- Errors (invalid JSON) are caught and handled safely
- Parsed data are then : auto-filled into frontend forms (CV parsing), Stored in the database (applications) & Logged into monitoring tables (activity logs)

**Token Limitation and Input Control**

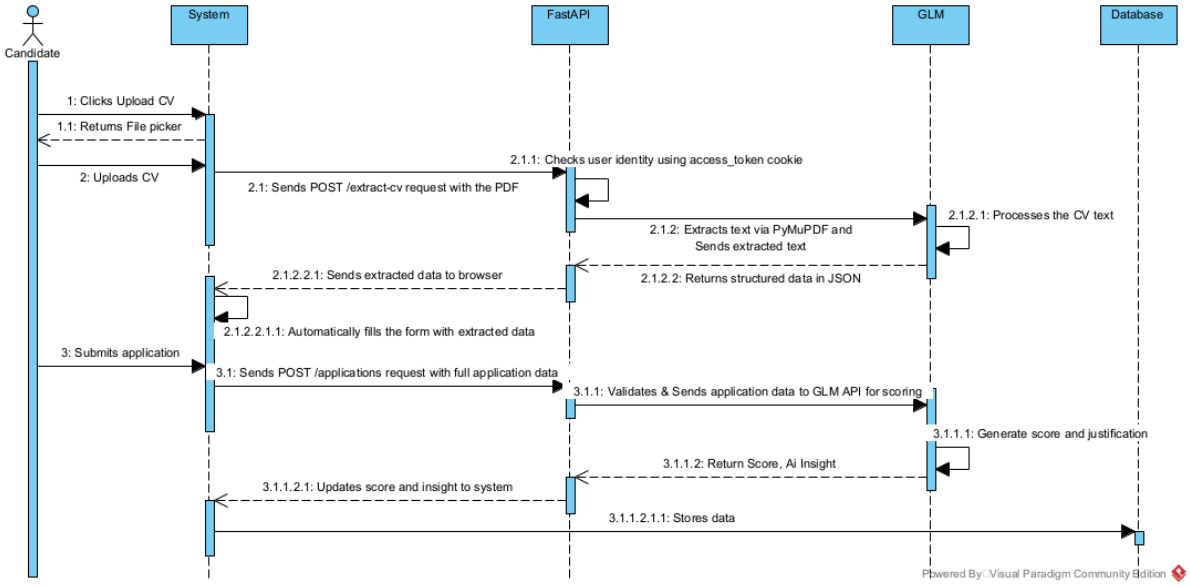
Token usage is controlled through:

- Input truncation (raw_text[:4000])
- Fixed prompt templates
- Structured output enforcement (response_format = json_object)
- Avoidance of unnecessary conversational context
- This prevents : API failures due to token overflow, Increased latency & Cost inefficiencies

### 2.1.3 Dependency Diagram




### 2.2.2. Sequence Diagram




a. Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]

This sequence diagram shows the full journey of a candidate uploading a CV and submitting a job application.

The process starts when the candidate interacts with the system by uploading a CV. The system processes the CV using the backend and AI service (GLM), extracts relevant information, and auto-fills the application form. The candidate then

reviews the data and submits the application. The backend evaluates the candidate using AI scoring and stores the final data in the database.

b. Show the interactions step by step.
   - The Candidate clicks the upload CV button
   - The System (Frontend) opens the file picker
   - The Candidate selects and uploads a PDF CV
   - The System sends a request to FastAPI (/extract-cv)
   - FastAPI verifies the user using the access token
   - FastAPI extracts text from the PDF using PyMuPDF
   - FastAPI sends the extracted text to GLM API
   - GLM processes the CV and returns structured JSON data
   - FastAPI sends the extracted data back to the System
   - The System automatically fills the form fields
   - The Candidate reviews and edits the form
   - The Candidate submits the application
   - The System sends application data to FastAPI (/applications)
   - FastAPI validates the user again
   - FastAPI sends the data to GLM for scoring
   - GLM returns a score and justification
   - FastAPI stores the application and AI results in the Database

c. Do demonstration of each key system features does operate from start to finish

## 2.2. Technological Stack

### 2.2.1. Frontend

- HTML, CSS, JavaScript
- Runs in browser (localhost / deployed web client)
- Handles:
  - Authentication (login/signup)
  - File upload (CV)
  - Dashboard visualization (HR analytics)
- Communicates with backend via REST API (Fetch/Axios)

### 2.2.2. Backend

- FastAPI (Python)
- RESTful API architecture
- Responsibilities:
  - Authentication & session management (Supabase Auth + cookies)
  - CV parsing endpoint (/extract-cv)
  - Application submission & tracking
  - HR dashboard analytics
  - AI interaction (GLM via OpenRouter)
  - Logging & monitoring

### 2.2.3. Database

- PostGRESQL (Supabase)
- Stores:
  - Users (profiles)
  - Applications (applications)

- Status tracking (application_status)
- Logs (activity_logs)

2.2.4. Cloud/Deployment

- Supabase
- Auth
- PostgreSQL DB
- Backend can be deployed on: “AWS EC2 / Render / Railway”
- External API: OpenRouter (GLM API gateway)

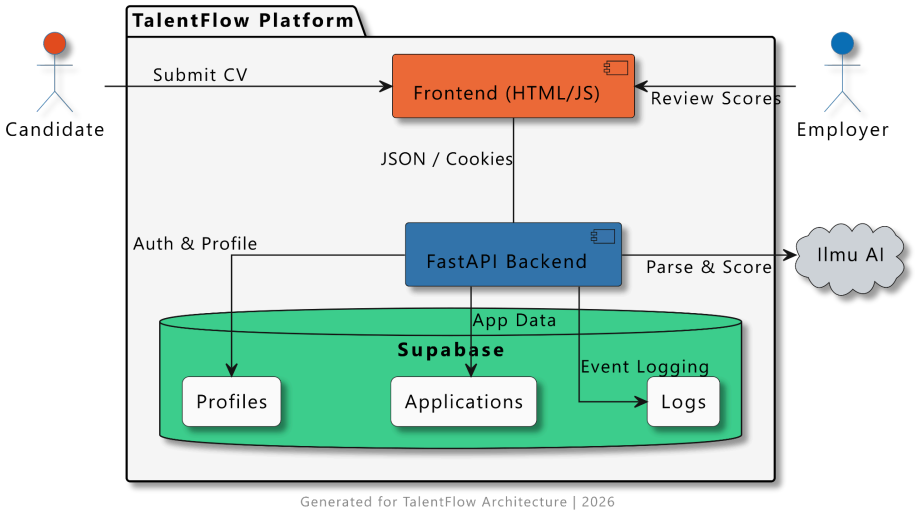
2.2.5. AI / LLM Layer

- **Z.AI GLM-4.5 (via OpenRouter API)**
- Used for: CV parsing → structured JSON, Candidate evaluation scoring, System logging insights

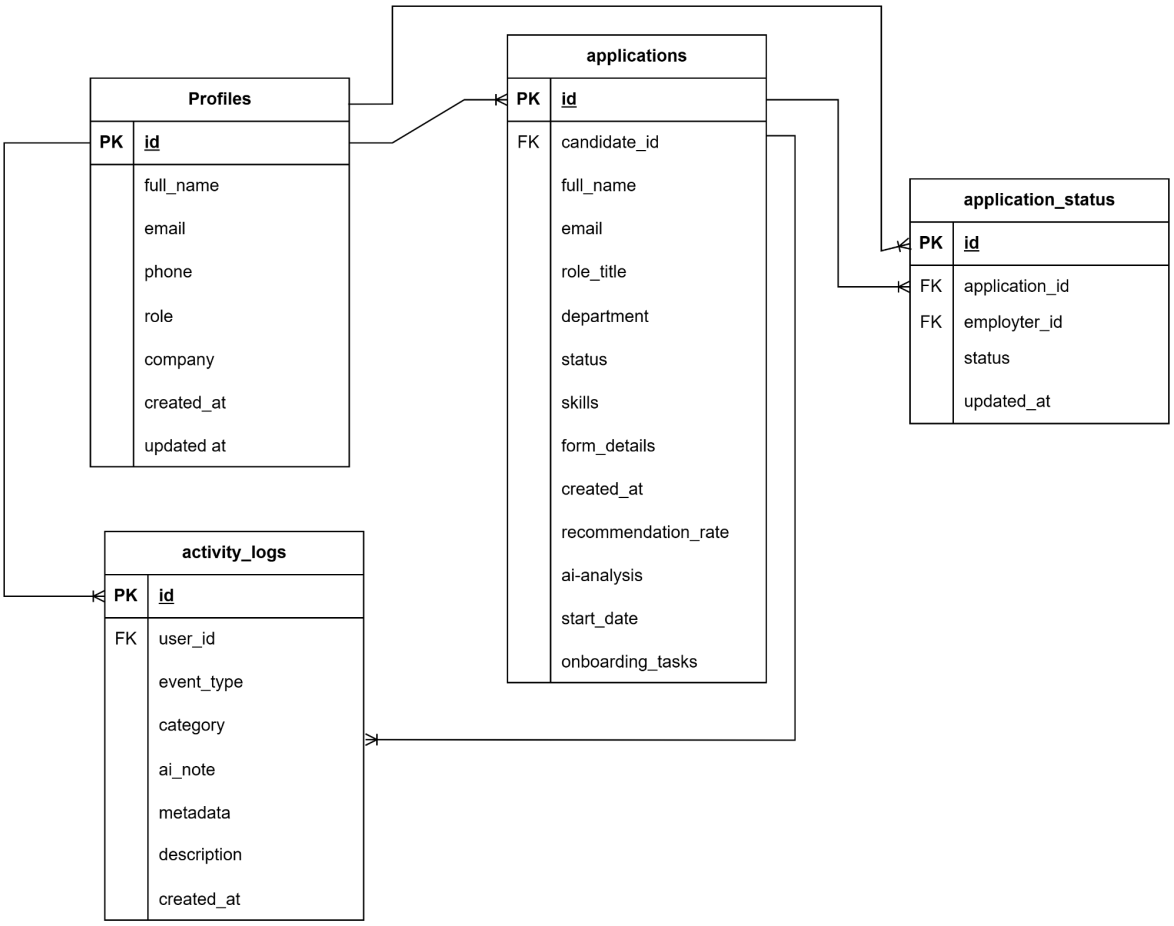
## 2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

***2.3.1.1. Data Flow Diagram (DFD)***



### 2.3.2. Normalized Database Schema



## 3. Functional Requirements & Scope

This section defines the system boundaries and core functionality of TalentFlow. It ensures that the development team focuses on a **high-impact Minimum Viable Product (MVP)** that demonstrates technical feasibility within the constraints of UMHackathon 2026.

### 3.1. Minimum Viable Product:

#### 3.1.1. The following core features will be implemented and demonstrated :

| # | Features | Description |
|---|---|---|
| **1** | Candidate Application Submission | Candidates submit structured applications through digital forms, including personal details, resume, and required documents. |
| **2** | HR Dashboard & Analytics | HR users access a centralized dashboard displaying application statistics, pipeline stages, department breakdowns, and recent applications. |
| **3** | AI Candidate Evaluation (GLM Integration) | Z.AI GLM processes candidate submissions to extract key information and generate a suitability score for hiring decisions. |
| **4** | Recruitment Pipeline Tracking | Candidates are tracked across different stages (e.g., New, Reviewing, Interview, Offer, Onboarding) with real-time updates |
| **5** | Onboarding and Start Date Monitoring | HR can track upcoming candidate start dates through a list view and calendar interface for onboarding management. |
| **6.** | AI assisted autofill | Users can simply upload their CV and the GLM may extract key notations from their cv to fill up the application where applicable |

### 3.2. Non-Functional Requirements (NFRs)

These define system qualities that ensure performance, reliability, and scalability.

| Quality (e.g.) | Requirements (e.g.) | Implementation (e.g) |
|---|---|---|
| Scalability | The system must handle increasing numbers of candidate applications without performance degradation. | Use Supabase (PostgreSQL) with scalable cloud infrastructure; backend APIs designed to handle concurrent requests. |
| Reliability | The system should maintain high availability and ensure no data loss during candidate submission or updates. | Backend validation before database writes; error handling and retry mechanisms for API calls. |

| | | |
|---|---|---|
| Maintainability | API responses (including AI processing) should return within acceptable time (≤ 3 seconds under normal load). | Asynchronous API calls; loading states in UI; timeout handling for GLM responses. |
| Performance | [e.g. The API calls to GLM must return a response within 3s under normal load but at p95] | [e.g. Timeout of 5s for Async API calls. Displaying user-facing loading state while awaiting for response] |
| Maintainability | The system should be modular and easy to update or extend. | Separation of frontend, backend, and AI logic; clean API structure; reusable UI components |
| Usability | The platform should be intuitive for both HR and candidates. | Clean dashboard UI, structured forms, visual charts, and clear navigation. |
| Security | Sensitive candidate data must be protected. | Authentication (session-based), secure API endpoints, and controlled database access. |
| Cost Efficiency | AI usage should remain within reasonable cost limits. | Limit prompt size, cache repeated AI results, and avoid redundant API calls. |

### 3.3. Out of Scope / Future Enhancements

The following features are part of the long-term vision but are **not included in the MVP for UMHackathon 2026**:

- Real-time messaging system between HR and candidates
- Automated interview scheduling with calendar integrations (e.g., Google Calendar)
- Advanced AI features (e.g., personality analysis, video interview analysis)
- Multi-company / multi-tenant support
- Notification system (email/SMS alerts for application updates)
- Resume parsing for multiple file formats with deeper semantic analysis
- Role-based access control with fine-grained permissions

## 4. Monitor, Evaluation, Assumptions & Dependencies

TalentFlow adopts a **controlled deployment strategy** to minimize risk when introducing new features:

**4.1. Technical Evaluation:**

4.1.1. **Grayscale Rollout & A/B Testing**:

- New features (e.g., AI scoring improvements, dashboard analytics updates) are first released to a **small subset (~5–10%) of HR users**
- System performance, error rates, and user interaction are monitored
- If stable, rollout is gradually expanded to **100% of users**
- Different feature variations (e.g., AI scoring display format, dashboard layout) are tested
- Metrics evaluated: "HR decision speed, Engagement with insights, Feature usability"
- The better-performing variant is selected for full deployment

4.1.2. **Emergency Rollback (ER) & Golden Release**:

- Golden Release
  - A stable, previously tested version of the system is maintained as a fallback
- Emergency Rollback (ER)
  - Triggered when critical issues are detected (e.g., API failure, incorrect AI scoring, data inconsistency)
  - System automatically reverts to the Golden Release version
  - Minimizes downtime and prevents data corruption

**4.1.3. Priority Matrix:**

4.1.3.1. Defines severity levels, trigger conditions, and response actions

| **Priority** | **Condition (trigger)** | **Action** |
|---|---|---|
| P1 – Critical | System unavailable, database failure, or AI scoring returns invalid/null for majority of requests (>20%) | Immediate Emergency Rollback to Golden Release, halt new deployments |
| P2 – High | API latency > 5s consistently, or dashboard fails to load key data | Scale backend services, restart affected services, monitor closely |
| P3 – Medium | Minor UI issues, incorrect visualizations, or delayed updates | Fix in next patch release, no rollback required |
| P4 – Low | Cosmetic issues or non-critical feature bugs | Logged for Future Improvement |

4.2. **Monitoring**:

4.2.1. TalentFlow incorporates a monitoring strategy to ensure system health:

"TalentFlow implements a centralized monitoring strategy to maintain overall system

health and performance. The backend continuously tracks API response times, error rates, and request volume, triggering alerts when thresholds such as high latency or abnormal error rates are exceeded. The database layer is monitored for query performance, connection stability, and data consistency to prevent failed writes or corruption.

On the AI side, the GLM integration is observed for response time, success rates, and token usage, with fallback mechanisms activated if repeated failures occur. The frontend layer monitors user-facing issues such as failed data loads or empty dashboard states, displaying error indicators when necessary.

All monitoring signals are consolidated through a logging and alerting mechanism that notifies developers in real time, enabling quick intervention before issues escalate into critical failures."

**4.3.** **Assumptions:** The following assumptions were made during development:

- Users (HR and candidates) have **stable internet connectivity**
- Candidates provide **accurate and complete application data**
- HR users regularly access the dashboard for monitoring
- AI model (GLM) returns **consistent and structured outputs**
- System usage is within expected scale during MVP stage
- Organizations have basic digital infrastructure to adopt the platform
-

4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

| **Tools** | **Purpose** | **Risks** |
|---|---|---|
| Z Ai GLM API | Core AI engine for resume analysis, data extraction, and candidate scoring | **High Risk** – API downtime or rate limits can disrupt AI features → Mitigation: fallback logic and cached results |
| Supabase (PostgreSQL) | Database for storing candidate data, applications, and analytics | **Medium Risk** – service outage affects data access → Mitigation: retries and error handling |
| Chart.js | Data visualization for dashboards (charts, graphs) | **Low Risk –** UI-only impact if it fails |
| FastAPI Backend | Handles API requests and business logic | **Medium Risk** – backend failure impacts entire system |
| Browser Environment (Client-side JS) | Runs frontend logic and UI rendering | **Low Risk -** depends on user device/browser compatibility |

## 5. Project Management & Team Contributions

The development of TalentFlow was executed within a 10-day sprint as part of UMHackathon. The approach focused on rapid prototyping, iterative refinement, and final validation to ensure both functionality and presentation quality.

### 5.1. Project Timeline:

| Phase | Duration | Activities | Justification |
|---|---|---|---|
| Prototype Creation | 17–19 April (3 Days) | Built core pages (Login, Homepage, Candidate, Dashboard), initial UI design, and basic frontend-backend integration | Establishes a working MVP early to allow time for improvements |
| Stress Testing & Refinement | 20–21 April (2 Days) | UI/UX improvements, input validation, feature adjustments (form simplification, monitoring page), bug fixing | Ensures system stability and improves user experience before finalization |
| Documentation & Pitch Deck | 22–25 April (4 Days) | Finalize backend integration, prepare system documentation, create pitch deck | Ensures clarity in presentation and technical explanation for judges |
| Submission | 26 April | Final system submission | Marks completion of development and deliverables |

### 5.2. Team Members Role:

| Member | Role | Responsibilities |
|---|---|---|
| Grace Wong Xin En | Frontend & UI Lead | Developed login page, implemented CSS styling, ensured consistency across light/dark modes |
| Lim Jing Jie | Account & Monitoring Developer | Built account management page and AI monitoring system |
| Abhishek Prakesh | Frontend Feature Developer | Developed homepage dashboard and candidate pages, including analytics and UI interactions |

| | | |
|---|---|---|
| Oh Kuan Qi | Backend & Database Lead | Implemented backend logic (Python), managed Supabase database, handled API integration |

### 5.3. Recommendations:

| **Area** | **Recommendation** | **Benefit** |
|---|---|---|
| Performance | Implement caching (e.g., Redis) for frequently accessed dashboard data | Reduces load time and database queries |
| Scalability | Transition to microservices architecture for backend components | Allows independent scaling of AI, API, and database services |
| Reliability | Add advanced monitoring and automated alerting systems | Improves system stability and faster issue detection |
| AI Integration | Implement stronger fallback mechanisms for | Ensures system remains functional even if AI service is unavailable |
| Security | Enhance authentication and security within data storage | Protects sensitive candidate data |